

# Projeto Flavia Panos

Explicacao simples da estrutura, tecnologias usadas e principais logicas implementadas.

**Resumo:** este projeto e uma aplicacao Angular para apresentar produtos artesanais

em uma vitrine e permitir que uma area administrativa cadastre, edite e remova

produtos e categorias. Os dados sao salvos em um arquivo `db.json` usando `json-`

`server` como backend local.

## 1. Objetivo do Projeto

---

O objetivo e criar uma vitrine online para os produtos da marca Flavia Panos. A parte publica mostra categorias, novidades, trabalhos e uma area de contato. A parte administrativa permite gerenciar os produtos que aparecem na vitrine.

O projeto possui dois lados principais:

- **Front-end:** feito em Angular, responsavel pelas telas e interacoes.
- **Back-end local:** feito com `json-server`, que le e grava dados no arquivo `db.json`.

## 2. Tecnologias Utilizadas

---

- **Angular 19:** framework usado para criar a aplicacao web.
- **TypeScript:** linguagem usada nos componentes, servicos e regras da aplicacao.
- **HTML e CSS:** usados para estruturar e estilizar as telas.
- **Angular Router:** usado para navegar entre Home, Trabalhos, Login e Admin.
- **HttpClient:** usado para fazer requisicoes HTTP para o backend local.
- **json-server:** simula uma API REST usando o arquivo `db.json`.

- **Signals do Angular:** usados para controlar estados como formulario, carregamento, mensagens e erros.

### 3. Como Rodar o Projeto

---

O projeto precisa de dois terminais abertos.

#### Terminal 1: iniciar o backend local

```
npm run api
```

Esse comando executa o `json-server` na porta `3000`, usando o arquivo `db.json`.

#### Terminal 2: iniciar o front-end

```
npm start
```

Esse comando inicia o Angular em modo de desenvolvimento, normalmente em `http://localhost:4200`.

### 4. Estrutura Geral do Projeto

---

Os arquivos principais ficam dentro da pasta `src/app`.

- `app.routes.ts`: define as rotas da aplicacao.
- `produto.service.ts`: concentra as chamadas HTTP relacionadas a produtos.
- `categoria.service.ts`: concentra as chamadas HTTP relacionadas a categorias.
- `auth.service.ts`: controla login, logout e protecao da area administrativa.
- `home`: tela inicial da aplicacao.
- `trabalhos`: tela com vitrines separadas por categoria.
- `vitrine`: componente reutilizavel que mostra produtos em cards.
- `login`: tela de acesso ao painel administrativo.
- `gerenciar-produtos`: painel administrativo com CRUD de produtos e categorias.

## 5. Banco de Dados Local

---

O arquivo `db.json` funciona como banco de dados local. Ele possui duas coleções principais:

```
{
  "categorias": [
    { "id": 1, "categoria": "Panos de prato" },
    { "id": 2, "categoria": "Tapete" },
    { "id": 3, "categoria": "Amigurumi" },
    { "id": 4, "categoria": "Kit" },
    { "id": 5, "categoria": "Meu benzinho" }
  ],
  "produtos": []
}
```

A relação entre produto e categoria é feita pelo campo `categoriaId`. Por exemplo, se o produto tem `categoriaId: 1`, ele pertence a categoria com `id: 1`.

Atualmente o banco foi deixado sem produtos cadastrados, usando `"produtos": []`. Isso faz com que o sistema comece limpo: as categorias continuam disponíveis, mas a vitrine e o painel administrativo só passam a mostrar produtos depois que o administrador cadastra novos itens.

## 6. Rotas da Aplicação

---

As rotas definem qual componente aparece para cada caminho da URL.

```
export const routes: Routes = [
  { path: 'trabalhos', component: TrabalhosComponent },
  { path: 'admin', component: LoginComponent },
  {
    path: 'gerenciarProdutos',
    component: GerenciarProdutosComponent,
    canActivate: [adminGuard]
  },
  { path: '**', component: HomeComponent }
];
```

A rota `gerenciarProdutos` possui `canActivate`, ou seja, só pode ser acessada se o guard permitir.

## Menu da pagina publica

O menu principal foi simplificado e nao possui mais a opcao SOBRE. Ele agora possui apenas os links principais da pagina:

```
<li><a href="/">HOME</a></li>
<li><a href="/#novidades">NOVIDADES</a></li>
<li><a href="/#contato">ENCOMENDA</a></li>
```

A logica e simples: o link `/` leva para a pagina inicial; `/#novidades` leva direto para a secao de novidades; e `/#contato` leva para a area de encomenda/contato.

## 7. Conexao do Front com o Back

A conexao com o backend e feita pelos services. Eles usam o `HttpClient` para buscar, criar, editar e remover registros.

### ProdutoService

```
private readonly apiUrl = 'http://localhost:3000/produtos';

listarProdutos(): Observable<Produto[]> {
  return this.http.get<Produto[]>(this.apiUrl);
}

criarProduto(produto: CriarProdutoPayload): Observable<Produto> {
  return this.http.post<Produto>(this.apiUrl, produto);
}
```

A ideia e simples: se a tela precisa de produtos, ela chama o service. O service faz a requisicao HTTP e devolve os dados para o componente.

## 8. Vitrine de Produtos

O componente de vitrine recebe uma categoria e busca produtos dessa categoria. Ele mostra cada produto em um card com imagem e nome.

```
<div class="card" *ngFor="let item of itens">
  <div class="img-container">
    <img [src]="item.imagem" alt="">
```

```
</div>
<p>{{ item.nome }}</p>
</div>
```

O ponto mais importante é o `[src]="item.imagem"`. Como o campo `imagem` pode guardar uma URL ou um Base64, o navegador consegue exibir a imagem diretamente.

## Botao de encomenda pelo WhatsApp

O botão `Encomendar` da Home e o ícone do WhatsApp na seção de contato abrem uma conversa no WhatsApp com número e mensagem já preenchidos.

```
<a
  href="https://wa.me/5511920083982?
text=0l%C3%A1%2C%20gostaria%20de%20encomendar%20um%20produto."
  target="_blank"
  rel="noopener noreferrer"
>
  Encomendar
</a>
```

O número usado é `+55 11 92008-3982`. Na URL do WhatsApp ele fica sem símbolos e espaços: `5511920083982`. A mensagem precisa estar codificada na URL para que acentos, vírgula e espaços sejam enviados corretamente.

## 9. CRUD Administrativo

---

CRUD significa criar, listar, atualizar e excluir. No projeto, o painel administrativo permite fazer isso para produtos e categorias.

### Listagem

Quando o painel abre, o método `carregarDados()` busca produtos e categorias ao mesmo tempo usando `forkJoin`.

```
forkJoin({
  produtos: this.produtoService.listarProdutos(),
  categorias: this.categoriaService.listarCategorias()
}).subscribe({
  next: ({ produtos, categorias }) => {
```

```
this.produtos.set(produtos);
this.categorias.set(categorias);
}
});
```

A vantagem é que a tela só atualiza quando as duas buscas terminam. Assim o painel consegue mostrar produtos junto com os nomes das categorias.

## Cadastro e Edição de Produto

O mesmo formulário serve para criar e editar produto. A diferença é o campo `id`. Se existe `id`, o produto já existe e será editado. Se não existe, será criado.

```
const requisicao = form.id
  ? this.produtoService.editarProduto(form.id, payload)
  : this.produtoService.criarProduto(payload);
```

## Remoção

Antes de remover um produto, o sistema pede confirmação com `window.confirm`. Se o usuário confirmar, o service chama o endpoint de exclusão.

## 10. Upload de Imagem em Base64

---

Antes, o formulário tinha um campo para digitar a URL da imagem. Agora ele usa upload de arquivo. O arquivo selecionado é convertido para Base64 no front-end e salvo no campo `imagem` do produto.

### Campo no formulário

```
<input
  type="file"
  accept="image/*"
  name="imagemProduto"
  (change)="selecionarImagem($event)"
>
```

O atributo `accept="image/*"` indica que o campo deve aceitar arquivos de imagem. Quando o usuário escolhe um arquivo, o Angular chama `selecionarImagem($event)`.

## Logica de conversao

```
selecionarImagem(event: Event) {
  const input = event.target as HTMLInputElement;
  const arquivo = input.files?.[0];

  if (!arquivo) {
    return;
  }

  if (!arquivo.type.startsWith('image/')) {
    this.erro.set('Selecione um arquivo de imagem valido.');
```

```
    input.value = '';
    return;
  }

  const leitor = new FileReader();

  leitor.onload = () => {
    this.atualizarProdutoCampo('imagem', String(leitor.result));
    this.limparAvisos();
  };

  leitor.readAsDataURL(arquivo);
}
```

A logica funciona assim:

- O usuario seleciona uma imagem.
- O codigo pega o primeiro arquivo selecionado.
- Se o arquivo nao for imagem, mostra erro.
- O `FileReader` le o arquivo.
- `readAsDataURL` transforma a imagem em uma string Base64.
- Essa string e colocada no campo `imagem` do formulario.
- Ao salvar, o Base64 vai para o `db.json`.

## Imagem obrigatoria

O produto so pode ser salvo se tiver nome, categoria e imagem. A validacao da imagem acontece antes de montar o payload.

```
if (!form.imagem.trim()) {
  this.erro.set('Selecione uma imagem para o produto.');
```

```
  return;
}
```

```
}
```

Isso garante que todo produto cadastrado tenha imagem. Na edicao, se o produto ja possui imagem, ela continua no formulario. O usuario so precisa escolher outra imagem se quiser trocar.

## 11. Autenticacao do Admin

---

Foi implementada uma autenticacao simples no front-end. O usuario deve informar:

- **Usuario:** admin
- **Senha:** admin

### Service de autenticacao

```
login(usuario: string, senha: string) {  
  const credenciaisValidas =  
    usuario.trim() === 'admin' && senha === 'admin';  
  
  if (credenciaisValidas) {  
    localStorage.setItem(this.chaveSessao, 'true');  
  }  
  
  return credenciaisValidas;  
}
```

Quando o login e valido, o sistema salva uma chave no `localStorage`. Essa chave indica que o admin esta logado. A senha nao e salva.

### Guard da rota administrativa

```
export const adminGuard: CanActivateFn = () => {  
  const authService = inject(AuthService);  
  const router = inject(Router);  
  
  if (authService.estaAutenticado()) {  
    return true;  
  }  
  
  return router.createUrlTree(['/admin']);  
};
```



O guard é executado antes de abrir a rota administrativa. Se o usuário estiver autenticado, ele entra. Se não estiver, é redirecionado para a tela de login.

## 12. Mensagens de Erro e Carregamento

---

O painel usa signals para controlar mensagens e estados da tela:

- `carregando`: indica quando uma requisição está em andamento.
- `mensagem`: mostra mensagens de sucesso.
- `erro`: mostra mensagens de erro.
- `produtos` e `categorias`: guardam os dados vindos da API.

Signals deixam a tela sincronizada com o estado. Quando um signal muda, o Angular atualiza a parte correspondente do HTML.

## 13. Limitações Importantes

---

- A autenticação atual é apenas no front-end. Ela é suficiente para demonstração e uso local, mas não é segurança real de produção.
- O `json-server` é uma ferramenta de desenvolvimento. Para produção, o ideal seria uma API real com banco de dados.
- Imagens em Base64 aumentam bastante o tamanho do `db.json`. Para muitos produtos, o ideal seria salvar arquivos em um servidor ou serviço de armazenamento.
- O backend precisa estar rodando em `localhost:3000` para o front conseguir carregar e salvar os dados.
- Como o banco local está sem produtos cadastrados, a vitrine fica vazia até que novos produtos sejam adicionados pelo painel administrativo.

## 14. Fluxo Completo do Sistema

---

1. O usuário acessa o site.
2. A Home mostra categorias, novidades e contato.

3. O menu permite ir para o inicio, novidades ou encomenda usando /, /#novidades e /#contato.
4. O botao de encomenda e o icone do WhatsApp abrem uma conversa com a mensagem pronta.
5. A vitrine busca produtos no json-server. Se produtos estiver vazio, nenhum card de produto aparece.
6. O administrador acessa /admin.
7. Ele faz login com admin/admin.
8. O guard libera o acesso ao painel.
9. No painel, o admin cadastra ou edita produtos e categorias.
10. Ao enviar uma imagem, ela e convertida para Base64.
11. Ao salvar, os dados sao enviados para o json-server.
12. O json-server grava tudo no db.json.
13. A vitrine passa a exibir os dados atualizados.

## 15. Conclusao

---

O projeto esta organizado como uma aplicacao Angular simples, com separacao clara entre componentes visuais, services de acesso aos dados, rotas e regra de autenticacao. A logica principal consiste em buscar dados do json-server, controlar formularios com signals, converter imagens para Base64 e proteger o painel administrativo com uma verificacao local de login.

Documento gerado para explicar a implementacao do projeto Flavia Panos.